

Secure File Storage System using AES Encryption

Introduction

With the increasing need for data privacy, securing sensitive files has become essential. Unauthorized access, data breaches, and tampering are common threats in digital environments. This project focuses on building a Secure File Storage System that uses encryption techniques to protect files from unauthorized access. The system ensures both confidentiality and integrity by encrypting files and verifying their authenticity during retrieval.

Objectives:

The main objectives of this project are:

- To securely encrypt and store files
- To allow safe decryption of stored files
- To ensure file integrity using hashing
- To provide a simple and user-friendly CLI interface
- To demonstrate practical implementation of cryptography

Technologies Used:

- Python
- cryptography (Fernet for AES encryption)
- hashlib (SHA-256 hashing)
- json (metadata storage)
- os (file handling)

System Design:

The system consists of two main modules:

Encryption Module

- Reads the input file
- Encrypts the data using AES (Fernet)
- Saves the encrypted file with .enc extension
- Generates metadata containing file details and hash

Decryption Module

- Reads the encrypted file

- Decrypts using the stored key
- Reconstructs the original file
- Verifies file integrity using hash comparison

Key Features:

File Encryption

Files are encrypted using AES-based Fernet encryption, ensuring data confidentiality.

File Decryption

Encrypted files can be safely restored using the same secret key.

Integrity Verification

SHA-256 hashing ensures that the file has not been altered during storage or transfer.

Metadata Storage

Stores important information such as original filename and hash for verification.

Robust Input Handling

Supports copy-paste and drag-and-drop file paths, improving usability.

Working Principle:

1. User selects a file for encryption
2. System generates or loads a secret key
3. File is encrypted and stored
4. Metadata file is created
5. During decryption:
 - File is decrypted
 - Hash is recalculated
 - Integrity is verified

Results

The system successfully encrypts and decrypts files while ensuring data integrity. It prevents unauthorized access and detects any tampering with stored files.

```
Windows PowerShell
PS D:\Internship\Projects_(FEB)\Secure_File_Storage_System> cat .\File.txt
Hai Buddy!!
This is a top secret file. Don't share with any one!!
PS D:\Internship\Projects_(FEB)\Secure_File_Storage_System> python File_Secure.py

Secure File Storage System
1. Encrypt File
2. Decrypt File
Choose option: 1
Enter file path: D:\Internship\Projects_(FEB)\Secure_File_Storage_System\File.txt
File encrypted: D:\Internship\Projects_(FEB)\Secure_File_Storage_System\File.txt.enc
PS D:\Internship\Projects_(FEB)\Secure_File_Storage_System> python .\File_Secure.py

Secure File Storage System
1. Encrypt File
2. Decrypt File
Choose option: 2
Enter .enc file path: D:\Internship\Projects_(FEB)\Secure_File_Storage_System\File.txt.enc
Integrity Verified
File decrypted: decrypted_File.txt
PS D:\Internship\Projects_(FEB)\Secure_File_Storage_System> cat .\decrypted_File.txt
Hai Buddy!!
This is a top secret file. Don't share with any one!!
PS D:\Internship\Projects_(FEB)\Secure_File_Storage_System>
```

Conclusion

This project demonstrates the practical use of encryption and hashing techniques to secure file storage. It provides a reliable and simple solution for protecting sensitive data.